



Programmierung eines Authoring-Tools (Dashboardeditor)

PROJEKTARBEIT

des Studienganges TIT22
an der Dualen Hochschule Baden-Württemberg Ravensburg

von
Eric Erath

09.10.2023

Bearbeitungszeitraum	12 Wochen
Matrikelnummer, Kurs	1910897, TIT22
Ausbildungsfirma	FPT Systems GmbH, Amtzell
Betreuer der Ausbildungsfirma	Jochen Geschwentner
Gutachter der Dualen Hochschule	Eric Balogh

Erklärung

Ich versichere hiermit, dass ich meine Projektarbeit mit dem Thema:

Programmierung eines Authoring-Tools (Dashboardeditor)

selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Ich versichere zudem, dass die eingereichte elektronische Fassung mit der gedruckten Fassung übereinstimmt.

Ort

Datum

Unterschrift

Inhaltsverzeichnis

Erklärung	2
Inhaltsverzeichnis	3
1. Projektbeschreibung	4
1.1 Allgemeines.....	4
1.2 Zielsetzung	4
2. Praktische Umsetzung.....	5
2.1 Bereits vorhandener Code.....	5
2.2 Hinzugefügte Features	7
2.2.1 Log-Filter	7
2.2.2 Automatisches Neuladen des Dashboards	8
2.2.3 Konfigurierbare Dashboardgrößen.....	9
2.2.4 Konfigurationsdatei über Explorer auswählbar	11
2.2.5 Konfigurationshilfen.....	13
2.2.5.1 Implementation	13
2.2.5.2 Mögliche Verbesserungen	16
2.2.6 Positionierung von Dashboardelementen mit graphischer Oberfläche.....	16
2.2.6.1 Hintergrund und Requirements	16
2.2.6.2 Implementation	18
2.2.6.3 Mögliche Verbesserungen	23
2.2.7 Umbenennung von OPCUA-Variablen im Editor	24
3. Ausblick	26
4. Zusammenfassung	27

1. Projektbeschreibung

1.1 Allgemeines

Die FPT Systems GmbH hat sich, als Tochterfirma der FPT Robotics GmbH & Co. KG, auf das Programmieren von Roboteranlagen der Schwesterfirma spezialisiert. Neben dem Erstellen von SPS-Projekten, die die mechanische Steuerung der Anlage und einen großen Teil des Bedienerschutzes übernehmen, zudem entwickelt die FPT Systems GmbH die Benutzeroberfläche FlexOP/GESTICA (fortan FlexOP). Diese Software übernimmt die Aufgaben, die nicht trivial von der SPS in der Roboteranlage zu lösen sind. Diese Aufgaben umfassen unter anderem Zugriff auf Netzwerke, eine Graphische Oberfläche zum Konfigurieren der Anlage, Anzeigen wichtiger Betriebsdaten und Ausgeben von Fehlermeldungen.

Zur Darstellung der Roboterdaten werden sogenannte Dashboards verwendet, diese sind über XML-Dateien konfigurierbar.

1.2 Zielsetzung

Im Rahmen der praktischen Tätigkeit soll ein Editor entwickelt werden, der den Konfigurationsprozess der Dashboards erleichtern soll. Aktuell wird das Add-In zur Darstellung von Dashboards mit XML-Dateien konfiguriert. Das Testen und Anpassen der Anzeigequalität in der Benutzeroberfläche der Software FlexOP/GESTICA ist aktuell sehr zeitaufwändig und schwierig, mitunter da zum Testen der Anzeige jedes Mal ein Neustart des Programms nötig ist. Ein Editor mit grafischer Oberfläche soll diese Anpassungen vereinfachen und beschleunigen.

2. Praktische Umsetzung

2.1 Bereits vorhandener Code

Zu Beginn des Projektes war bereits eine große Menge Code vorhanden, die einen Teil von FlexOP simulieren konnte, sodass es bereits möglich war die XML-Konfigurationsdatei einer Dashboardseite zu laden. So konnten Konfigurationen bereits spürbar schneller vorgenommen werden, da es nicht mehr nötig war das gesamte Softwarepaket FlexOP neu zu starten.

FlexOP ist mithilfe des .Net-Frameworks programmiert. Innerhalb dieses Frameworks bietet der System.Windows.Forms-Namespace ein intuitives und einfaches System Applikationen mit graphischer Benutzeroberfläche zu programmieren. Die Klasse System.Windows.Forms.Control bildet den Grundbaustein, sie können wie ein eigenes Fenster angezeigt werden und weitere Objekte der Klasse Control, wie beispielsweise Knöpfe oder Textfelder enthalten.

Die Klasse FxControl ist eine, von der FPT Systems GmbH geschriebene, Unterklasse der System.Windows.Forms.Control. Sie wird genutzt, um ein Dashboard darzustellen.

```
namespace FxA_Dashboard
{
    /// <summary>
    ///     This is a base GUI-FxAddIn-Page to display in FlexOP
    /// </summary>
    5 references
    public partial class FxControl : UserControl, IFlexOpUiAddIn, IFlexOpPage
    {
        private readonly IFxDependencyService _dependencyService = FxDependencyService.Instance;
        public string Configpath;

        Constructor
        Interfaces
    }
}
```

Abbildung 1, Implementation der Klasse FxA_Dashboard.FxControl

Die DashboardTestApp macht sich das Konzept der Polymorphie zunutze, denn anstatt das Dashboard in die Oberfläche von FlexOP einzubetten, wird es in ein leeres Fenster gelegt.

Die Daten, die eigentlich aus einer SPS über das OPC-UA Protokoll ausgelesen würden werden von der DashboardTestApp auch simuliert. Aufgrund des Protokollnamen werden diese Werte bei FPT im allgemeinen OPCUA-Variablen genannt. Zusätzlich wird dem Benutzer eine Tabelle angezeigt, in der Testwerte für die Variablen festgelegt werden können.

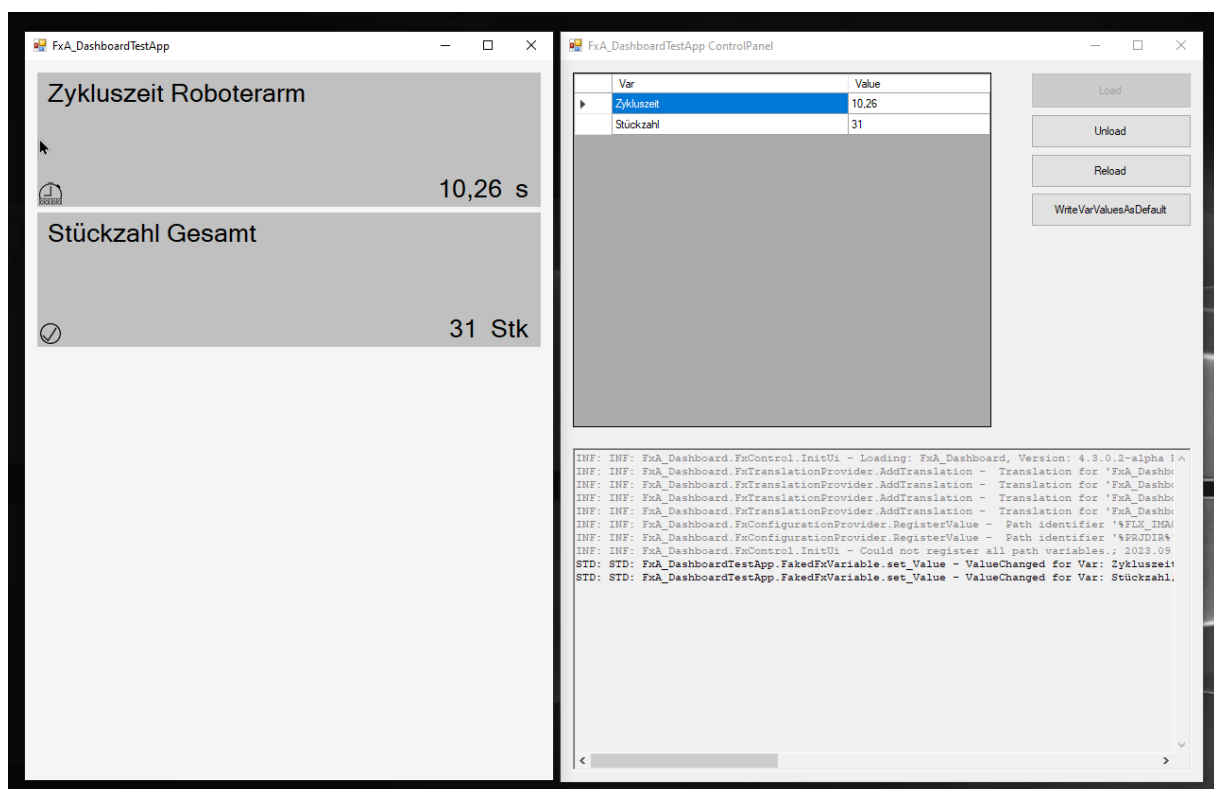


Abbildung 2, beispielhaftes Dashboard links, DashboardTestApp rechts

Die DashboardTestApp wird über die Praxisphase zu einem Editor erweitert. Dafür wird das Programm umbenannt zu FxA_DashboardEditor und auch im Folgenden als Dashboardeditor bezeichnet.

2.2 Hinzugefügte Features

2.2.1 Log-Filter

Der Dashboardeditor besitzt ein Textfeld, in dem Statusmeldungen des Dashboards, beispielsweise Fehlermeldungen oder Debugnachrichten, angezeigt werden. Bei Dashboard mit vielen Subelementen kommt es aber häufig dazu, dass das Meldungsfenster unübersichtlich wird. Daher wird dem Dashboardeditor eine Möglichkeit zum Filtern der Nachrichten gegeben.

Zu diesem Zweck besitzt die verwendete FxTrace-Library bereits eine Variable zum Setzen eines LogLevels. Das LogLevel ist die Dringlichkeit, die eine Nachricht mindestens haben muss, um im Nachrichtenfenster zu erscheinen.

Der Benutzer wählt mithilfe einer Combobox das gewünschte LogLevel aus, dabei wird die `comboBoxLogFilter_SelectedIndexChanged` Methode aufgerufen, in der die entsprechende Variable gesetzt wird.

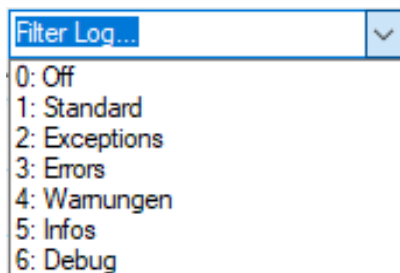


Abbildung 3, Combobox im Dashboardeditor

```
1 reference
private void comboBoxLogFilter_SelectedIndexChanged(object sender, EventArgs e)
{
    FxTrace.TracingProvider.LogLevel = (Fx.Framework.FxConstants.LogLevel) comboBoxLogFilter.SelectedIndex;
}
```

Abbildung 4, Methode zum Handeln des SelectedIndexChanged-Events

2.2.2 Automatisches Neuladen des Dashboards

Nachdem ein Anlagenprogrammierer Änderungen an der Konfigurationsdatei vorgenommen hat und diese Testen will, ist ein manuelles Klicken auf den Reload-Knopf nötig. Da allerdings nur selten ein Aktualisieren des Dashboards nach speichern der XML unerwünscht ist, soll der Dashboardeditor automatisch die Anzeige aktualisieren.

Hierzu erhält der Dashboardeditor einen Timer, sodass in regelmäßigen Abständen geprüft wird, ob eine neue Version der Konfigurationsdatei gespeichert wurde. Einmal pro Sekunde wird geprüft, ob der letzte Speicherzeitpunkt der XML-Datei gleich dem ist, der beim letzten Reload gespeichert wurde. Wenn nicht, bedeutet das, dass Änderungen an der Konfiguration vorgenommen wurden und ein Neuladen des Dashboards wird ausgelöst.

```
1 reference
private void timerAutoReload_Tick(object sender, EventArgs e)
{
    DateTime xmlConfigDate = File.GetLastWriteTime(_dashboardConfigPath);

    if (!Equals(xmlConfigDate, _xmlConfigDate))
    {
        reload_Form();
        _xmlConfigDate = xmlConfigDate;
    }
}
```

Abbildung 5, Methode die Zyklisch auf Updates der Konfiguration prüft

Für den Fall, dass der zuständige Anlagenprogrammierer keinen Automatischen Reload des Dashboards will, wird zudem eine Checkbox zum Ein- und Ausschalten des Features implementiert. Ähnlich wie die Combobox aus 2.2.1 löst auch die Checkbox ein Event aus. Die verlinkte Methode startet und stoppt den Timer nach Bedarf.

2.2.3 Konfigurierbare Dashboardgrößen

Aus einem Feedback von Thomas Blümel, Teamleiter Software-Operations der FPT Systems GmbH, ging hervor, dass aus Sicht der Anlagenprogrammierer, die größte Schwäche des Dashboardeditors ist, dass angezeigte Dashboardseiten nicht dieselben Dimensionen haben wie in FlexOP. Außerdem kann in FlexOP ein Meldungsfenster eingeschaltet werden, was die Größe des Angezeigten Dashboards verändert.

Zu diesem Zweck wird der XML-Konfigurationsdatei des Dashboardeditors, ein neuer Abschnitt zum Festlegen sogenannter SizePresets hinzugefügt.

```
<!--DashboardSizePresets-->
<DashboardPreset>
  <ConfigScreenSizePreset Name="KukaSmartPad" Width="600" Height="800" />
  <ConfigScreenSizePreset Name="FxNeutro" Width="1920" Height="1080" />
</DashboardPreset>
```

Abbildung 6, SizePresets in der FxA_DashboardEditor.config.xml

Diese Konfigurationen werden beim Start des Editors ausgelesen und in einer Liste gespeichert. Als Minimalkonfiguration wird zudem ein Preset „Resizable“ hinzugefügt. Sollte dieses Preset ausgewählt werden, so kann der Benutzer wie von Windows gewohnt mithilfe der Maus selbst die Größe des Dashboards einstellen.

```
private List<List<string>> sizePresets = new List<List<string>>()
{
    new List<string>() { "Resizable", "0", "0" }
};
```

Abbildung 7, Deklaration der sizePresets-Liste

Beim Start der Anwendung werden die Presets ausgelesen und in die Liste gespeichert, zusätzlich werden die Namen einer Combobox hinzugefügt.

```
foreach (ConfigScreenSizePreset preset in _config.DashboardPreset)
{
    sizePresets.Add(new List<string>() { preset.Name, preset.Width, preset.Height });
}

foreach (List<string> preset in sizePresets)
{
    comboBoxScreenSize.Items.Add(preset[0]);
}
```

Abbildung 8, Ausschnitt aus dem Code zum Laden der Konfiguration des Dashboardeditors

Wenn der Anwender aus der Combobox eine Größe auswählt, wird zunächst versucht den gespeicherten String zu einem Integer zu parsen, sollte dabei ein Fehler auftreten wird der Benutzer darauf hingewiesen, dass der angegebene Wert aus der XML fehlerhaft ist und das Dashboard erhält die Standardgröße 600x800 Pixel. Können die Dimensionen richtig gelesen werden wird als nächstes geprüft, ob Höhe und Breite gleich null sind, in diesem Fall handelt es sich um ein „Resizable“-Preset, also startet das Dashboard auch mit Standardgröße, zusätzlich wird aber die Möglichkeit zum Verändern der Fenstergröße freigeschalten. Zuletzt wird geprüft, ob die Dimensionen negativ sind, in diesem Fall erhält der Anlagenprogrammierer eine Fehlermeldung, ansonsten startet das Dashboard mit der spezifizierten Auflösung.

Zusätzlich zu Konfigurierbaren Fenstergrößen wird auch eine Checkbox zum Simulieren des aktivierten Meldungsfensters hinzugefügt. Wird diese Option aktiviert, reduziert der Editor die Höhe des Dashboards um 160 Pixel, die Größe des Meldungsfensters. Wird ein Preset mit variabler Größe verwendet, wird das Meldungsfenster automatisch deaktiviert und die Checkbox unsichtbar gemacht, da eine Möglichkeit zum Simulieren des Meldungsfensters in diesem Fall nicht benötigt wird und so der Editor etwas übersichtlicher bleibt.

2.2.4 Konfigurationsdatei über Explorer auswählbar

Zum Laden der Konfigurationsdatei des Dashboards, benötigt der Dashboardeditor den Dateipfad mit Name der Datei. Bisher ist der Pfad im Code festgelegt, als: „[.]bin\FxAddInTestApps\FxA_DashboardTestApp\net472\TestData\test1.config.xml“ „[..]“ steht hier für den Ordner, in dem sich das VisualStudio-Projekt des Dashboardeditors befindet.

Eine nutzerfreundliche Alternative bietet die OpenFileDialog Klasse. Mithilfe dieser Klasse ist es möglich den Anwender in der gewohnten Umgebung des File-Explorers Dateien auszuwählen.

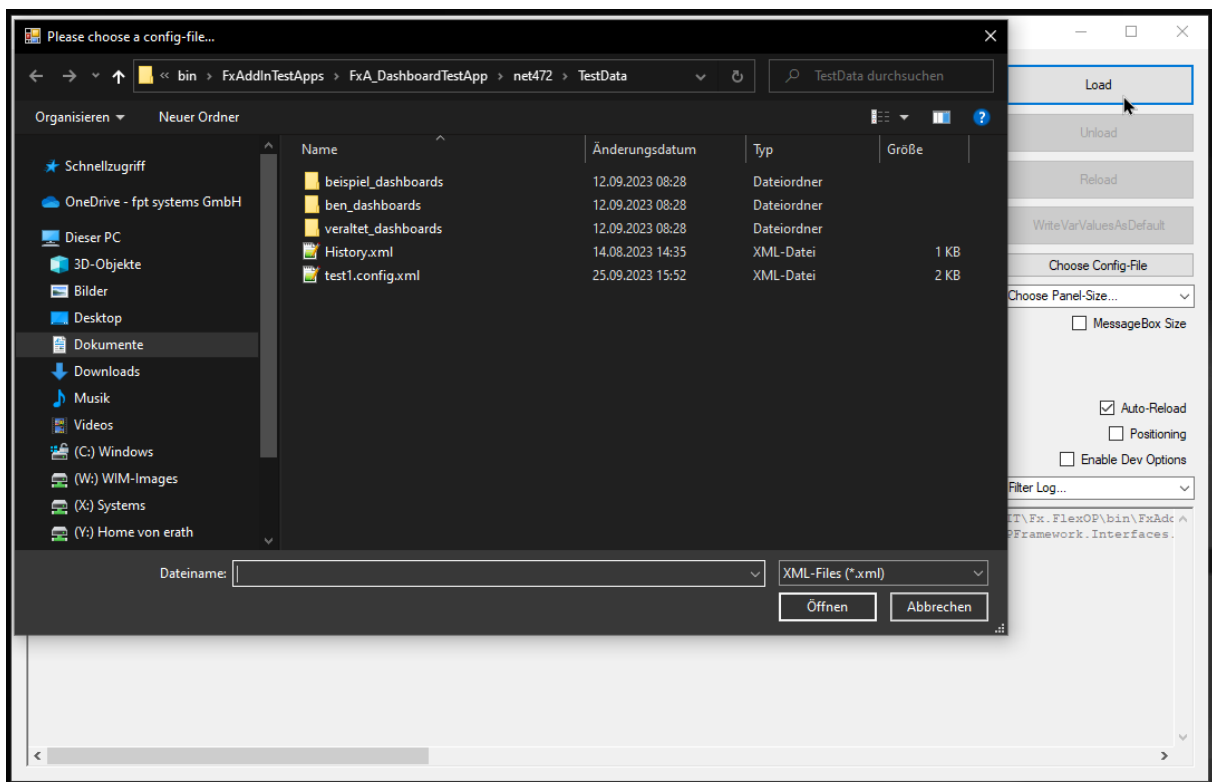


Abbildung 9, Nutzer wird aufgefordert eine Konfigurationsdatei auszuwählen

Nach dem Initialisieren des OpenFileDialogs lässt sich über einen Filter festlegen welche Dateiformate angezeigt werden sollen. Ein solcher Filter hat das Format:

„(Name)|(Dateiendung1);(Dateiendung2)...“. Groß- und Kleinschreibung wird bei der Dateieindung ignoriert. Wenn dem Benutzer die Wahl zwischen mehreren Filtern gegeben werden soll, werden die Strings der verschiedenen Filter, von senkrechten Strichen getrennt, concateniert.

Obwohl der Dashboardeditor jede Datei im ASCII-Format, die die XML-Syntax verwendet, unabhängig von der Dateieindung, erfolgreich laden kann, wird der Filter eingestellt, nur Dateien anzuzeigen, die auf „.xml“ enden anzuzeigen. Dies hat die Gründe, dass es einerseits Pest-Practice ist einen Hinweis auf das Dateiformat in der Endung anzugeben, und andererseits wird dadurch die Dateiauswahl übersichtlicher.

```
this.openConfigFileDialog.Filter = "XML-Files|*.xml";  
this.openConfigFileDialog.Title = "Please choose a config-file...";  
  
this.openConfigFileDialog.FileOk += new System.ComponentModel.CancelEventHandler(this.openConfigFileDialog_FileOk);
```

Abbildung 10, Konfiguration des OpenFileDialogs

Wenn der Anwender seine Auswahl bestätigt, wird mithilfe des Events „FileOk“ die Methode `openConfigFileDialog_FileOk` aufgerufen (sh. Abb. 10). In dieser Methode wird der Dateipfad der Konfigurationsdatei in die Variable `_dashboardConfigPath` gespeichert.

Um eine Konfigurationsdatei auszuwählen, muss der Anlagenprogrammierer nur auf den neuen Knopf mit der Aufschrift: „Choose Config-File“ klicken. Da der Dateipfad jetzt nicht mehr von Anfang an gesetzt ist, ist es möglich, dass ein Benutzer auf den „Load“-Knopf drückt, bevor ein Valider Pfad gesetzt ist, das würde zu unerwünschtem Verhalten des Editors führen, also wird beim Klick auf den Knopf überprüft, ob schon ein Wert für die Variable `_dashboardConfigPath` gesetzt wurde. Sollte das nicht der Fall sein, wird dem Anwender der Dialog zu auswählen einer Datei angezeigt.

```
3 references
private void btnLoad_Click(object sender, EventArgs e)
{
    if (string.IsNullOrEmpty(_dashboardConfigPath))
    {
        openConfigFileDialog.ShowDialog();
    }

    if (!string.IsNullOrEmpty(_dashboardConfigPath))...
```

Abbildung 11, Fehlerbehandlung beim Klicken auf „Load“-Knopf

Solange der Explorer geöffnet ist, wird der Code im Dashboardeditor automatisch pausiert und nach Beendigung an derselben Stelle fortgesetzt. Daher ist es nicht nötig in der Methode openConfigFileDialog_FileOk zu prüfen welches der auslösende Knopf war und dementsprechend das Dashboard zu laden.

2.2.5 Konfigurationshilfen

2.2.5.1 Implementation

Einige Subelemente bieten den Anlagenprogrammierern spezielle Funktionen an, um beim Konfigurieren zu helfen. Diese Konfigurationshilfen, oder DevOptions, werden bisher in der XML-Datei ein und aus geschaltet. Beispielsweise kann die DashboardScreenConfig über das Boolean „ShowGridLines“ das Raster anzeigen lassen, an dem die einzelnen Controls des Dashboards ausgerichtet werden.

Weitere DevOptions bieten die DashboardChartControl, zum manuellen hinzufügen von Messwerten, und das DashboardMapElement, zum Anzeigen der Koordinaten innerhalb des Bildes.

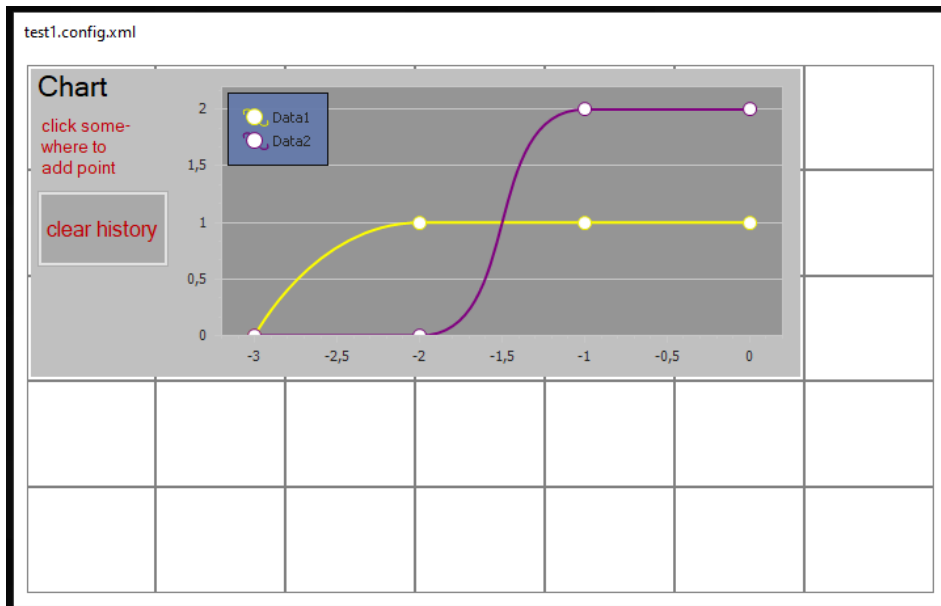


Abbildung 12, Dashboard mit und aktivierten DevOpts

Auf Wunsch von Harald Gläss, Geschäftsführer der FPT Systems GmbH, sollen diese Tools nicht mehr über die Konfigurationsdatei aktivierbar sein. Grund dafür ist mitunter, dass diese Hilfestellungen im laufenden Anlagenbetrieb nicht benötigt werden, sondern nur beim Konfigurieren im Dashboardeditor. Daher ist der Editor perfekt geeignet eine grafische Oberfläche zum Einschalten der DevOptions bereitzustellen.

Da die Konfigurationshilfen von den Controls implementiert werden, aber aus dem Editor aktiviert werden sollen, muss es in jeder Control, die diese Möglichkeiten anbieten soll, eine öffentliche („public“) Methode geben diese zu aktivieren beziehungsweise zu deaktivieren. Für erhöhte Einheitlichkeit wird diese Methode „DevOpts“ genannt und erhält als Input ein Boolean, über das die eigentliche Steuerung stattfindet.

Die Methode loadDevOpts, wird im Dashboardeditor implementiert und iteriert in einer foreach-Schleife über alle Controls des Dashboards. In diesem Loop wird für jedes Subelement geprüft, ob es sich um eines handelt, das DevOptions anbietet und dementsprechend wird die DevOptions-Methode aufrufen. Der zu übergebende

Boolean wird aus dem Zustand der neuen „Enable Dev Options“-Checkbox (sh. Abb. 9: rechter Rand) abgeleitet.

```
5 references
private void loadDevOpts()
{
    bool devopt = checkDevOpt.Checked;
    foreach (Control item in _frmMain.Controls)
    {
        if (item is DashboardMapElement)
        {
            (item as DashboardMapElement).Dev_Options(devopt);
            (item as DashboardMapElement).Disable_Navigations(devopt);
        }
        else if (item is DashboardScreenControl)
        {
            (item as DashboardScreenControl).Dev_Options(devopt);
        }
        else if (item is DashboardChartElement)
        {
            (item as DashboardChartElement).Dev_Options(devopt);
        }
    }
}
```

Abbildung 13, Methode zum Aktivieren der DevOpts

Im Fall des MapElements wird zudem noch die Methode Disable_Navigations aufgerufen. Wie Dev_Options funktioniert auch diese über das Inputboolean und aktiviert beziehungsweise deaktiviert die sogenannten Navigations. Einige Controls im Dashboard können den Anwender duck klicken auf andere Dashboards navigieren, diese Funktion ist im Editor nicht verfügbar, kann aber dennoch getestet werden, indem stattdessen ein Popupfenster mit einer Meldung, dass navigiert wurde erscheint. Diese Meldungen werden durch Disable_Navigations gesteuert. Dies ist nötig da die Konfigurationshilfe der Control klicken involviert und die Popups der Navigations beim Testen stören.

2.2.5.2 Mögliche Verbesserungen

Ein Mangel dieses Lösungsansatzes ist, dass neu implementierte Controls, die auch DevOpts anbieten sollen, manuell zu der Methode im DashboardEditor hinzugefügt werden müssen. Dies lässt sich lösen, indem ein Interface „IHasDevOpts“ definiert wird. Jede Klasse, die diese Interface Implementieren soll muss eine Version Methode Dev_Options mit Inputboolean besitzen. Die Methode im Editor prüft dann nur noch, ob die Subelemente das Interface „IHasDevOpts“ implementieren. So muss ein Programmierer, der neue Controls mit Konfigurationshilfen programmiert nur das Interface in seinem Code implementieren.

Zusätzlich muss noch das MapElement leicht umprogrammiert werden, sodass Disable_Navigations automatisch beim Aufruf von Dev_Opts aufgerufen wird.

2.2.6 Positionierung von Dashboardelementen mit graphischer Oberfläche

2.2.6.1 Hintergrund und Requirements

Wie bereits in 2.2.5.1 erwähnt, verwendet das Dashboard ein Raster, an dem die Subelemente ausgerichtet werden, dies erspart den Anlagenprogrammierern die Mühe jede Position und Größe in Pixeln anzugeben. Die DashboardScreenConfig macht dieses Feature möglich, indem es als Grundelement des Dashboards in der Konfigurationsdatei alle anderen elemente in der Liste „Elements“ enthält und beim Laden durch diese Liste aus Controls iteriert, wobei jedes Element im TableLayoutPanel „_panel“ angeordnet wird und so Position und Größe in Pixeln automatisch aus den angegebenen Werten in Tabellen-Koordinaten berechnet werden.


```
foreach (IDashboardBaseElementConfig dashboardElementConfig in config.Elements)
{
    IDashboardElementControl element;
    if (dashboardElementConfig is IDashboardDateTimeConfig) ...
    else if (dashboardElementConfig is IDashboardClockConfig) ...
    ...
    else if (dashboardElementConfig is IDashboardRangeGaugeConfig) ...
    else ...

    if (element is DashboardBaseElementControl dashboardBaseElementControl) ...

    result = result && element.SetParameter(dashboardElementConfig, strConfigPath, flexOpApp);
    if (element is Control newDashboardControl)
    {
        newDashboardControl.Dock = DockStyle.Fill;
        _panel.Controls.Add(newDashboardControl, element.GridPosition.X, element.GridPosition.Y);
        _panel.SetColumnSpan(newDashboardControl, element.TileSize.X);
        _panel.SetRowSpan(newDashboardControl, element.TileSize.Y);
    }
}
```

Abbildung 14, Ausschnitt aus der SetParameter-Methode der DashboardScreenControl

Die automatische Anordnung in einer Tabelle bietet den Vorteil, von leichter verständlichem Code und übersichtlicheren Konfigurationsdateien, hat aber den Nachteil, dass Subelemente des Dashboards sich nicht gegenseitig überlappen können. Wenn es zu einer Überlagerung kommen würde, zwingt Microsofts Implementation des TableLayoutPanel die Controls andere Positionen und Größen, ohne leicht erkennbares Muster. Dadurch entsteht ein relativ großes Problem für Anlagenprogrammierer, denn bereits ein Off-by-one-Error kann zu Platzkonflikten und somit zum Verschieben großer Teile des Dashboards führen. Wenn stattdessen die einzelnen Elemente an der ihnen zugewiesenen Stelle erscheinen würden, wäre es trivial falsch angegebene Positionen zu finden und beheben. Daher wäre eine Möglichkeit Dashboardelemente, zumindest im Editor, überlappen zu lassen sehr sinnvoll.

Da das TableLayoutPanel nicht nicht konfiguriert werden kann, Überschneidung von Elementen zuzulassen, wird als Alternative eine neue Funktion im Dashboardeditor hinzugefügt, die erlaubt Position und Größe von Controls mithilfe einer graphischen Oberfläche zu verändern.

Der sogenannte Positioning-Modus soll dem Nutzer erlauben eine Control des Dashboards, durch klicken auszuwählen und anschließend mithilfe von Knöpfen auf einer graphischen Oberfläche dessen Position und Größe zu verändern. Dabei sollen diese Änderungen in echtzeit auf dem Dashboard angezeigt werden. Für den Fall, dass durch verschiebungen im Dashboard die Controls nicht ihre vorgesehene Position haben soll zudem angezeigt werden wo sich das Element, laut XML-Datei, befinden sollte.

2.2.6.2 Implementation

Genau wie die DevOptions aus 2.2.5 wird auch der Positioning-Modus mithilfe einer Checkbox aktiviert. Zusätzlich wird implementiert, dass beide Modi nicht gleichzeitig aktiviert werden können, indem beim einschalten einer der Funktionen die Checkbox disabled, also nicht klickbar, wird. Für besseres Feedback an den Nutzer wird der Text dabei auch automatisch ausgegraut. Im Gegensatz zu den DevOpts ist der Positioner aber auch bei Reloads fehleranfällig, daher werden zudem die Knöpfe beziehungsweise Checkboxes „Unload“, „Reload“ und „Auto-Reload“ beim Start disabled.

Da der Positioner die Subelemente durch klicken auswählen soll, aber einige dieser Elemente eigenen Code, wie Navigations, ausführen, müssen diese Funktionen temporär deaktiviert werden. Dazu wird ähnlicher Code wie in 2.2.5.1 verwendet, der für alle Subcontrols mit Navigatoinen, diese abschaltet. Nebenher werden noch die DevOpts der DashboardSreenConfig eingeschaltet. Dies zeigt das, dem Dashboard zugrundeliegende, Raster an, was dem Nutzer bei der Positionierung behilflich sein kann. Nachdem das Klicken auf jegliche Controls keinen Effekt mehr hat, kann nun ohne Probleme das Click_Event im Dashboardeditor abgefangen und separat behandelt werden. Dazu wird die neue Methode subscribe_ClickEvent aufgerufen.

Diese Methode sucht zunächst das `TableLayoutPanel`, in dem alle Elemente des Dashboards eingebettet sind. Anschließend iteriert der Code über alle Controls in der Tabelle. Wenn das Inputboolean der Methode auf „True“ gesetzt ist, wird in der Schleife eingestellt, dass beim Klicken auf die Elemente die Methode `Dashboard_Click` aufgerufen werden soll, andernfalls wird das Event unsubscribed, also wird die Methode beim Click nicht mehr aufgerufen.

```
6 references
private void subscribe_ClickEvent(bool toggle = true)
{
    if (_frmMain != null)
    {
        foreach (Control item in _frmMain.Controls)
        {
            if (item is TableLayoutPanel table)
            {
                _table = table;
                foreach (Control element in _table.Controls)
                {
                    if (toggle)
                    {
                        element.Click += Dashboard_Click;
                    }
                    else
                    {
                        element.Click -= Dashboard_Click;
                    }
                }
            }
        }
        return;
    }
}
```

Abbildung 15, Methode „subscribe_ClickEvent“ des *Dashboardeditor*

Die `Dashboard_Click` Methode erhält als Input eine Referenz auf die angeklickte Control. Dabei kann es sich aber beispielsweise auch um ein Textfeld innerhalb einer `DashboardControl` handeln. Daher muss zunächst geprüft werden zu welchem Element das angeklickte Subelement gehört. Dies geschieht in einer weiteren Schleife

über alle Controls des Dashboards, in der abgefragt wird, ob das aktuelle Element „Parent“ von dem geklickten Element ist. Die Methode `IsParentOf` wird von der Basisklasse aller Controls implementiert und somit auch an alle weiteren vererbt. Diese Methode liefert als Ergebnis nur dann den Wert „True“, wenn das Input-Element in der Aufrufenden Control enthalten ist, oder es sich um das Selbe element handelt.

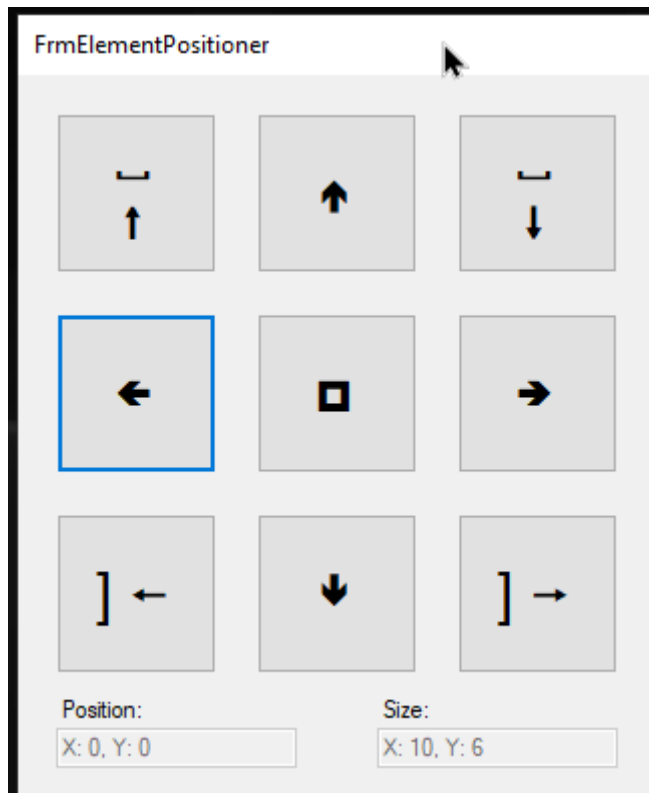
```
2 references
private void Dashboard_Click(object sender, EventArgs e)
{
    if (_frmPos != null)
    {
        if (_frmPos.IsDisposed)
        {
            _frmPos = null;
        }
    }
    if (checkPositioning.Checked && _frmPos == null)
    {
        UpdateControls();
        Control subelement = sender as Control;
        foreach (Control control in _table.Controls)
        {
            if (control.IsParentOf(subelement))
            {
                checkPositioning.Enabled = false;
                btnLoad.Enabled = false;
                btnUnload.Enabled = false;
                btReload.Enabled = false;
                btnWriteVarValues.Enabled = false;

                _frmPos = new FrmElementPositioner(control, _dashboardConfigPath, _table);
                _frmPos.Show();
                _frmPos.FormClosed += _frmPosClosedEvent;
            }
        }
    }
}
```

Abbildung 16, `Dashboard_Click` Methode

Nachdem das richtige Parent-Element gefunden wurde, werden zunächst die vonhin benannten Knöpfe deaktiviert, anschließend wird eine neue Instanz der Klasse `FrmElementPositioner` erstellt. Diese Klasse übernimmt die restlichen Aufgaben des Verschiebens.

Als Input im Konstruktor erhält die Instanz der Klasse FrmElementPositioner referenzen auf die Control, die zu verschieben ist, und auf das TableLayoutPanel in



dem sie enthalten ist, zudem auch noch einen Dateipfad zur XML-Konfigurationsdatei des Dashboards in String-Format. Zunächst zeigt der Positioner neun Knöpfe, in einem 3x3 Feld, an. Der Knopf in der Mitte speichert die Änderungen, die vier direkt angrenzenden Knöpfe verschieben die ausgewählte Control in die vier Kardinalen Richtungen und die Knöpfe in den Ecken kontrollieren die Dimensionen der Control.

Abbildung 17, Instanz der FrmElementPositioner-Klasse

Ein Klick auf die verschiedenen Knöpfe verändert, je nach Knopf, Position oder Größe der Control um ein Kästchen im Dashboard-Raster, dabei wird kontrolliert, dass die Control nicht die Grenzen des Dashboards überschreitet. Da alle Klassen in C# sogenannte Referencetypes sind, ändert das gleichzeitig die Werte in der Control auf dem Dashboard. Um diese Änderungen sichtbar zu machen müssen die Änderungen

zudem noch im TableLayoutPanel vorgenommen werden. Dazu stellt Microsofts Implementation die Methoden SetColumn, SetRow, SetColumnSpan und SetRowSpan bereit.

Nachdem den Anwender die nötigen Änderungen vorgenommen hat und auf den Speichern-Knopf drückt, sollen die Änderungen auch in der XML-Datei vorgenommen werden.

Dies stellt ein Problem dar, denn im Programm können Instanzen von Klassen eindeutig anhand ihrer Speicheradresse identifiziert werden, das ist in der Konfigurationsdatei nicht möglich. Als Lösung für dieses Hindernis wird der Basisklasse für Dashboardelemente eine ID als weiteres Attribut hinzugefügt, das auch in der XML steht. Somit haben dank Vererbung alle auf dem Dashboard verwendbaren Elemente das ID-Attribut. Die IDs in der XML-Datei werden noch vor dem Laden des Dashboards im Dashboardeditor überschrieben und durch eine vortlaufende Nummerierung ersetzt, somit muss der Anlagenprogrammierer nicht für jedes Element die IDs selbst setzen, keine zwei Elemente können die selbe ID haben und jede Control des geladenen Dashboards kann über die ID eindeutig ein Abschnitt der Konfigurationsdatei zugewiesen werden.

Beim Speichervorgang des Positioners wird zunächst die Konfigurationsdatei geladen. Daraus wird die Liste aller Elemente extrahiert und das Element ausgewählt, dessen ID mit der der ausgewählten Elements übereinstimmt. Ein Szenario, in dem kein Passendes Element gefunden wird, oder zu viele, sollte dank der Automatischen vergabe der IDs unmöglich sein, dennoch werden beide Fehlerfälle abgefangen um, im Worst-Case, einen Programmabsturz zu verhindern. Nachdem das Richtige Element gefunden wurde, werden die Attribute TileSize und GridPosition aktualisiert und die XML-Datei neu gespeichert.

```
1 reference
private void write()
{
    XElement DashboardXML = XElement.Load(_configPath);

    XElement ElementsXML = DashboardXML.Element("Elements");

    IEnumerable<XElement> possible_Elements = ElementsXML.Elements()
        .Where(element => element.Attribute("ID").Value == _elementControl.ID)
        .Select(element => element);

    if (possible_Elements.Count() > 1) ...
    else if (possible_Elements.Count() != 1) ...
    else
    {
        possible_Elements.First().Attribute("GridPosition").Value = $"{{X={_posX},Y={_posY}}}";
        possible_Elements.First().Attribute("TileSize").Value = $"{{X={_sizeX},Y={_sizeY}}}";
        DashboardXML.Save(_configPath);
    }
}
```

Abbildung 18, Methode zum Speicher der Änderungen im Positioner

2.2.6.3 Mögliche Verbesserungen

Da die Navigationen ähnlich gesteuert werden, wie die DevOps aus 2.2.5.1 treten auch hier dieselben Schwächen auf. Also wäre auch hier die Lösung über ein Interface „IHasNavigations“ oder ähnliches sinnvoll. Alternativ bietet sich im Fall des Positioners noch eine Alternative mithilfe der Vererbung an. Dem Basiselement der Dashboardcontrols kann eine Methode „TogglePosOpts“ hinzugefügt werden, die an alle weiteren Controls vererbt wird. Controls, die eigenen Code ausführen sollen, wenn der Positioning-Modus aktiviert wird, beispielsweise das Abschalten von Navigationen, können die geerbte TogglePosOpts überschreiben.

Eine weitere Schwäche des Codes ist das Konzept der ID, durch das Bearbeiten der XML-Datei mithilfe des „System.xml.linq“-Namespace gehen Kommentare, die sich in der Konfigurationsdatei, aber außerhalb der XML-Struktur befinden, verloren. Dazu kommt zudem, wenn vor den Änderungen ein invalider Pfad für die XML-Datei angegeben wurde, hat die DashboardTestApp automatisch eine gültige

Konfigurationsdatei am angegebenen Pfad erstellt und zudem Kommentaren über die verwendbaren Elemente und deren Attribute hinzugefügt.

Eine Dokumentation dieses Features, wenn es überhaupt eine Dokumentation gibt, ist nicht auffindbar. Auch der Nutzen ist eher eingeschränkt, da jede implementierte Control für das Dashboard im Intranet der FPT-Firmengruppe bereits dokumentiert ist. Daher wurde der Verlust diese Features bisher in Kauf genommen. Eine Möglichkeit Kommentare in der XML-Datei beizubehalten wäre dennoch eine dringende Verbesserung.

Eine letzte Optimierung gilt dem TableLayoutPanel. Eine eigene Implementation oder Tochterklasse der bereits verwendeten Version könnte Überlappungen von Controls zulassen. Dies wäre in Kombination mit dem Positioner die Optimale Lösung für das in 2.2.5.1 beschriebene Problem.

2.2.7 Umbenennung von OPCUA-Variablen im Editor

Das letzte hinzugefügte Feature geht aus einer Bemerkung von Jochen Geschwentner hervor. Im Dashboardeditor können für die simulierten OPCUA-Variablen Werte zum Testen festgelegt werden. Aufgrund der Tabelle fühlt es sich für den Benutzer intuitiv richtig an, dass dort nicht nur die Werte geändert werden können, sondern auch die Spalte mit den Variablennamen.

	Var	Value
	Zykluszeit	10,26
▶	TeileProduziert	23
	StatusOk	True

Abbildung 19, Tabelle zum Ändern der simulierten OPCUA-Variablen

Dazu wird die Methode, der Klasse DataGridVarProvider, die das Event behandelt, bei wenn sich ein Eintrag in der Tabelle ändert, also von Benutzer überschrieben wird. Bisher, war ein bearbeiten der ersten Spalte deaktiviert und in der Methode wurde nur sicherheitshalber geprüft, ob die Änderung in Spalte zwei vorgenommen wurde. Da jetzt auch Änderungen in der Spalte mit Index null möglich sind, wird zu beginn mithilfe von zwei If-Statements geprüft, in welcher der beiden Spalten des Event auftrat. Für den Fall, dass der Index eins ist, wird derselbe Code wie zuvor ausgeführt. Wenn aber der Index null ist, also der Name einer OPCUA-Variable geändert wurde, werden alle Vorkommen des alten Variablennamens, der aus der Liste `_variables` auszulesen ist, in der XML-Datei umbenannt zu dem neuen Namen.

```
1 reference
internal void OnCellValueChanged(object sender, DataGridViewCellEventArgs e)
{
    if (e.ColumnIndex == 0)
    {
        int nIndexRow = e.RowIndex;

        XElement DashboardXML = XElement.Load(_dashboardConfigPath);
        XElement ElementsXML = DashboardXML.Element("Elements");

        IEnumerable<XElement> possible_Elements = ElementsXML.Descendants()
            .Select(e => e)
            .Where(e => e.Attribute("DataReference")?.Value == _variables[nIndexRow].VarName);

        foreach (XElement element in possible_Elements)
        {
            try
            {
                _variables[nIndexRow].VarName = _dataGrid.Rows[nIndexRow].Cells[0].Value.ToString();
                element.Attribute("DataReference").Value = _dataGrid.Rows[nIndexRow].Cells[0].Value.ToString();
            }
            catch
            {
                FxTrace.LogErr("The datareference you were trying to enter was invalid.");
            }
        }

        DashboardXML.Save(_dashboardConfigPath);
    }

    if (e.ColumnIndex == 1)
    {
        int nIndexRow = e.RowIndex;

        FakedFxVariable var = _variables[nIndexRow];
        var.Value = _dataGrid.Rows[nIndexRow].Cells[e.ColumnIndex].Value;
    }
}
```

Abbildung 20, OnCellValueChanged-Methode des DataGridVarProvider

3. Ausblick

Eine logische Schlussfolgerung daraus, dass viele der Vorgestellten Änderungen am Dashboardeditor darauf abzielen, dass Anlagenprogrammierer weniger direkt in der XML-Datei Arbeiten müssen, wäre das Ziel den Editor zu einem vollständigen Editor, der für alle möglichen Änderungswünsche an der Konfigurationsdatei eine graphische Alternative bietet.

Ein solches Feature ist in der Tat möglich. Grundsätzlich ist dazu nötig, dass der Dashboardeditor eine Möglichkeit hat für jede Control eine Liste der Namen aller Parameter, die in der XML-Datei gesetzt werden können, erhält. Eine Ähnliche Funktion wird bereits standardmäßig in jedem Element implementiert. Beim Laden des Dashboards, wenn keine Konfigurationsdatei am angegebenen Pfad liegt, iteriert der Editor über alle implementierten Controls und erhält einen String, der als Kommentar in die automatisch generierte XML-Datei eingefügt wird. In diesem String stehen die Namen aller Parameter der Elemente sowie eine kurze Beschreibung der Funktion. Anstatt umständlich über String-Formatierung aus diesem Kommentar die Namen und Beschreibungen auszulesen kann jede Control über ein Interface eine neue Funktion „GetXmlParameters“ implementieren, die dem Dashboardeditor eine Liste mit Namen und Beschreibung übergibt.

Mithilfe dieser Methode und der in 2.2.6.2 beschriebenen Vorgehensweise zur Auswahl einer Control ist es möglich einen Editor zu implementieren der jede direkte Arbeit an der XML-Datei ersetzen kann, indem beim Klick auf eine Control ein neues Fenster erscheint, mit Textfeldern für alle Verfügbaren Parameter des Elements. Nachdem der Anwender die Änderungen über einen „Speichern“-Knopf bestätigt, werden die Änderungen mithilfe der Methoden aus dem „System.xml.linq“-Namespace in die Konfigurationsdatei übertragen und das Dashboard neu geladen damit die Änderungen sichtbar gemacht werden.

Der Geschäftsführer der FPT Systems GmbH, Harald Gläss, hat jedoch gefordert, dass ein solches Feature vorerst nicht implementiert wird. Einerseits, da vermutlich der Programmierungsaufwand dieser Features zu groß wäre, andererseits ist das Lesen und Bearbeiten von XML-Dateien eine Grundkenntnis für jeden Anlagenprogrammierer bei FPT und die Konfiguration von Dashboards sind ein guter Weg dies sicherzustellen und auch zu trainieren.

4. Zusammenfassung

Dashboardeditor wurde bereits erfolgreich für die Konfiguration mehrerer Dashboards im Rahmen einer Roboteranlage für die Fakuma-Messe konfiguriert. Wobei der zuständige Programmierer Andreas Jäger-Blankenburg stets Feedback zu den bestehenden Funktionen und Vorschläge für weitere Verbesserungen lieferte. Ob der Dashboardeditor jedoch die gewünschte Beschleunigung des Konfigurationsprozesses bietet, bleibt noch abzuwarten, da bei einem Großteil der Konfiguration der Dashboardeditor noch nicht fertiggestellt war und es zudem häufiger zu Verzögerungen kam. Beispielsweise funktionierte der verwendete Touch-Bildschirm nicht einwandfrei mit den Methoden zur Klick-Erkennung in C#.

Da nun eine fertiggestellte Version des Dashboardeditors in ein eigenes Repository des FPT-GitHubs verschoben wird, sollte die Verteilung an die restlichen Anlagenprogrammierer erleichtert werden. Mithilfe der größeren Menge an Testern sollte dann auch besser festzustellen sein, ob der Dashboardeditor seine Aufgaben erfüllt.

Ausgehend davon ist es sehr wahrscheinlich, dass noch neue Erweiterungen zum Dashboardeditor hinzugefügt werden. Im selben Zuge werden voraussichtlich einige der erwähnten Schwachstellen des Codes behoben werden.